

Space complexity
 ↳ if new array is created
 ↳ array, number
 O(1)

Time and space complexity
 (2, 0, 8)
 ↳ if new array is created
 ↳ array, number
 O(1)
 (1) O(1) + no loop, recursion
 (2) O(log n) -> iterating half
 (3) O(n log n) -> sorting
 (4) O(n) -> sorting
 (5) O(1) -> recursion
 (6) O(n) -> sorting
 (7) O(1) -> sorting

Searching

1. Linear search

- ↳ iterate over complete array
- ↳ search for an element in an array

2. Binary search

- ↳ sorted array
- ↳ find something in this array
- ↳ iterate half

[2, 3, 6, 8, 9, 10], target = 9
 l = 0, right = 5, mid = 2

while left <= right
 mid = (left + right) / 2
 if arr[mid] == target
 return mid
 else if target > arr[mid]
 left = mid + 1
 else right = mid - 1

l = 3
 right = 5
 mid = 4

target = 10
 [1, 3, 5, 7, 10, 12]
 l = 0, mid = 2, r = 5
 mid = 3, r = 5, result = 4, right = 3
 mid = 4, r = 3, result = 2
 Duplicate [2, 5, 2, 5], 2
 l = 0, r = 4, mid = 2, result = 2, r = 1
 while l <= r
 mid = (l + r) / 2
 if arr[mid] == target
 result = mid
 r = 1, l = 1

l = 0, right = 3
 mid = 1, right = 3
 arr[mid] < arr[right]
 left = 2, right = 2

Jump search

- ↳ sorted array
- ↳ search in an array
- ↳ skip

n = arr.length
 skip = int(sqrt(n))
 prev = 0
 segment
 prev = skip

Time = $\sqrt{n}$

[2, 3, 4, 5, 6, 10], target = 5
 prev = 0, skip = 2
 while prev < n and arr[min(skip, n) - 1] < target
 {
 prev = skip # 2
 skip = skip + int(sqrt(n)) # 4
 if (prev > n) break
 }
 linear search which prev = n and all found = target
 {
 prev = 2
 skip = 4
 }